



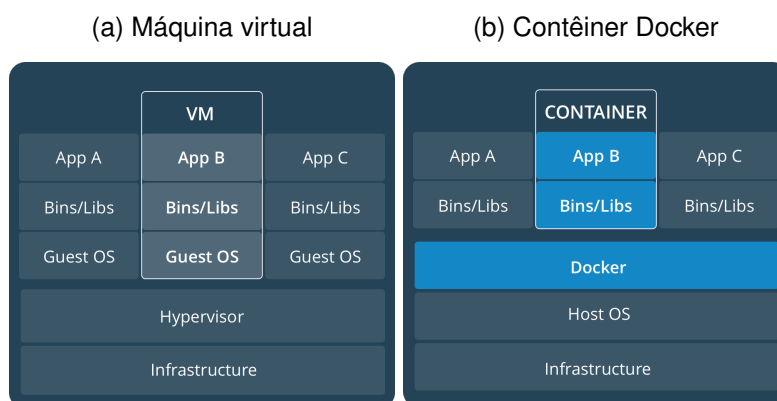
Laboratório 1: Docker

11/03/2026

1 Contêineres de software

Na Figura 1 é apresentado um comparativo entre máquinas virtuais e contêineres Docker. Uma máquina virtual requer um sistema operacional e o *hypervisor* virtualiza o acesso aos recursos de *hardware* do computador hospedeiro (*host*). Um contêiner Docker é executado de forma nativa no Linux e compartilha o kernel da máquina hospedeira com outros contêineres. Trata-se de uma solução mais leve, se comparada com a máquina virtual.

Figura 1: Contêineres vs máquinas virtuais



Fonte: docker.com

2 Docker

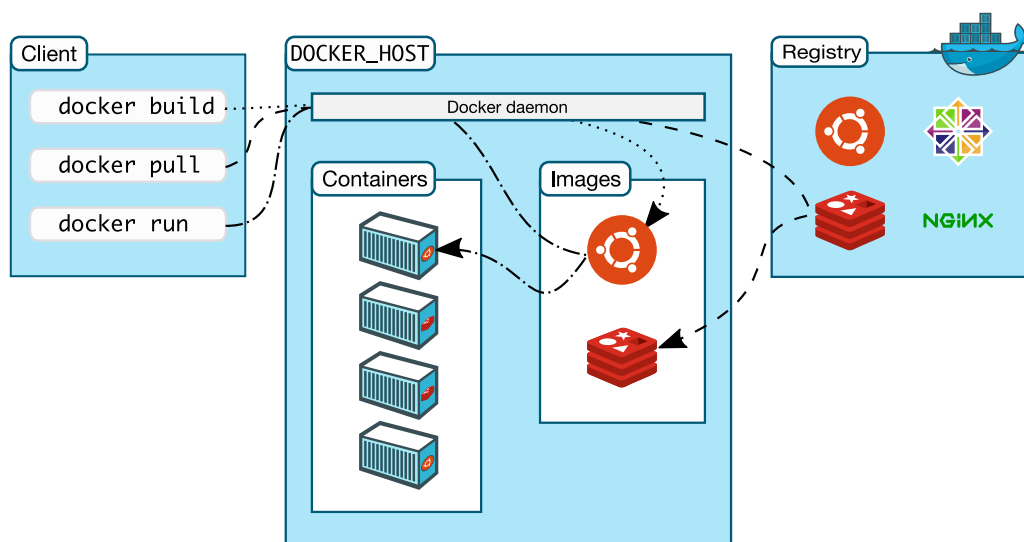
O **Docker Desktop** é uma aplicação que facilita a instalação e o uso do Docker em sistemas Windows, macOS e algumas distribuições Linux. Ele inclui o Docker Engine, o Docker Compose e outras ferramentas necessárias para criar, executar e gerenciar contêineres Docker em um ambiente gráfico amigável. Acesse <https://www.docker.com/products/docker-desktop/> para baixar o instalador.

Uma **imagem Docker** é um pacote somente leitura que contém tudo o que é necessário para executar uma aplicação: código-fonte, bibliotecas, dependências, variáveis de ambiente e instruções de configuração. Fazendo uma analogia com a programação orientada a objetos, a imagem seria uma classe e a instância dessa classe seria o **contêiner**. Assim como uma classe define um tipo de objeto, a imagem define um tipo de contêiner.

O **contêiner** é uma instância em execução da imagem, isolada do sistema operacional do host, mas compartilhando o kernel do sistema. Cada contêiner pode ser iniciado, parado, removido e até mesmo ter múltiplas instâncias sendo executadas simultaneamente a partir da mesma imagem. De acordo com a documentação oficial, um contêiner consiste de: (1) uma imagem; (2) um ambiente de execução; e (3) um conjunto padronizado de instruções. Ou seja, um contêiner Docker define uma forma padronizada para entregar *software*.

O **Docker Registry** é um serviço de armazenamento e distribuição de imagens Docker. Ele permite que os usuários publiquem, compartilhem e baixem imagens Docker. O Docker Hub (<https://hub.docker.com/>)

Figura 2: Arquitetura do Docker



Fonte: docker.com

//hub.docker.com) é o registro público mais popular, mas também existem registros privados e outros serviços de registro de terceiros, como o GitHub Container Registry (<https://ghcr.io/>) e o Amazon Elastic Container Registry (ECR) (<https://aws.amazon.com/ecr/>).

Na Tabela 1 são apresentados alguns comandos Docker para realizar tarefas comuns, como buscar imagens, executar contêineres e gerenciar os recursos do Docker. Para uma lista completa de comandos, acesse a <https://docs.docker.com/engine/reference/commandline/docker/>.

Tabela 1: Alguns comandos Docker

Exemplo	Descrição
<code>docker search ubuntu</code>	Busca imagens no Docker Hub
<code>docker pull eclipse-temurin:25-jdk</code>	Baixa uma imagem do Docker Hub
<code>docker images</code>	Lista todas as imagens locais
<code>docker run -it --rm ubuntu bash</code>	Executa um contêiner a partir de uma imagem
<code>docker ps</code>	Lista contêineres em execução
<code>docker ps -a</code>	Lista todos os contêineres (ativos e inativos)
<code>docker stop web</code>	Finaliza a execução de um contêiner de nome web
<code>docker start web</code>	Inicia um contêiner parado de nome web
<code>docker rm web</code>	Remove um contêiner parado de nome web
<code>docker container prune</code>	Remove todos os contêineres inativos
<code>docker exec -it web bash</code>	Executa um comando em um contêiner em execução de nome web
<code>docker logs web</code>	Exibe os logs de um contêiner de nome web
<code>docker build -t minha-imagem .</code>	Constrói uma imagem a partir de um Dockerfile

3 Executando algumas aplicações em contêineres

É possível executar um contêiner a partir de uma imagem e depois acessar o terminal interativo do contêiner para instalar pacotes, criar arquivos, etc. Veja o exemplo da Listagem 1, onde executamos um contêiner a partir da imagem do Ubuntu, redirecionamos o terminal interativo para o bash do contêiner e depois instalamos o aplicativo `net-tools` para usar o comando `ifconfig` dentro do contêiner e verificar o endereço IP que ele recebeu.

Listagem 1: Instalando pacotes em um contêiner Ubuntu

```
# Executando um contêiner a partir da imagem do Ubuntu
# Redirecionando o terminal interativo para o bash do contêiner
docker run -it --rm --name meu-ubuntu ubuntu bash

# Instalando o aplicativo nettools para usar o comando ifconfig dentro do contêiner
apt-get update && apt-get install -y net-tools

# Verificando o endereço IP do contêiner
ifconfig
```

⚠ Atenção

Ao executar um contêiner a partir de uma imagem, o Docker cria uma camada de leitura e escrita sobre a imagem somente leitura. Assim, qualquer modificação feita dentro do contêiner (como instalação de pacotes, criação de arquivos, etc) só afetará essa camada de leitura e escrita e não a imagem original. Se o contêiner for removido, todas as alterações feitas nele também serão perdidas, pois não afetam as camadas originais da imagem.

É possível mapear um diretório do computador hospedeiro para um diretório dentro do contêiner. Isso é útil para compartilhar arquivos entre o hospedeiro e o contêiner, por exemplo, arquivos de configuração, código-fonte, etc. Também é um mecanismo que se pode usar para persistir dados gerados dentro do contêiner mesmo após sua remoção. Veja o exemplo da Listagem 2, onde mapeamos o diretório local `dados` para um diretório `/dados` dentro do contêiner. Assim, qualquer arquivo criado ou modificado dentro do diretório `/dados` do contêiner também estará disponível no diretório local `dados` do hospedeiro.

Listagem 2: Mapeando um diretório do hospedeiro para o contêiner

```
# Criando um diretório na máquina hospedeira para ser mapeado para o contêiner
mkdir dados

# Executando um contêiner e mapeando o diretório local dados para o diretório /dados do contêiner
docker run -it --rm -v `pwd`/dados:/dados --name meu-ubuntu ubuntu bash

# Criando um arquivo dentro do diretório /dados do contêiner
echo "Olá, mundo!" > /dados/ola.txt
# Saindo do contêiner
exit

# Verificando que o arquivo criado dentro do contêiner também
# está disponível no diretório local dados do hospedeiro
cat dados/ola.txt
```

Quando um contêiner é executado, ele recebe um endereço IP privado dentro de uma rede virtual criada pelo Docker. Esse endereço IP é usado para comunicação entre os contêineres e o host, mas não é acessível diretamente a partir de outros dispositivos na rede local do hospedeiro. Para acessar serviços que estão sendo executados dentro do contêiner, é necessário redirecionar as portas do contêiner para as portas do computador hospedeiro.

Na Listagem 3 é apresentado um exemplo de execução de contêiner a partir da imagem do Nginx (um servidor web), onde redirecionamos a porta 8000 do computador hospedeiro para a porta 80 do contêiner. Assim, ao apontar o navegador web para `http://localhost:8000`, podemos acessar o servidor web do Nginx em execução dentro do contêiner.

Listagem 3: Executando um contêiner Nginx com redirecionamento de portas

```
# Executando um contêiner a partir da imagem do nginx,  
# Redirecionando a porta 8000 do hospedeiro para a porta 80 do contêiner  
# O parâmetro -d inicia o contêiner no modo detached, ou seja, em segundo plano  
docker run -d -p 8000:80 --rm --name web nginx:alpine
```

Para interromper a execução do contêiner, use o comando: `docker stop web`.

3.1 Exercício: Executando um servidor web simples dentro de um contêiner

Com base nos exemplos apresentados em Listagem 1, Listagem 2 e Listagem 3, execute um contêiner com a imagem do Nginx, mapeie um diretório local para o diretório `/usr/share/nginx/html` do contêiner e redirecione a porta 8000 do hospedeiro para a porta 80 do contêiner. Depois, crie um arquivo `index.html` dentro do diretório local mapeado com o conteúdo: `<html><h1>Meu Nome Completo</h1></html>` e acesse a URL `http://localhost:8000` no navegador web para ver o resultado.

4 Criando uma imagem personalizada

Ao utilizar o Docker, muitas vezes as imagens disponíveis nos repositórios públicos não atendem completamente às necessidades específicas de um projeto, seja por falta de determinados pacotes, configurações personalizadas ou ajustes de segurança. Nesses casos, criar uma imagem personalizada permite adaptar o ambiente de execução exatamente ao que a aplicação exige, garantindo maior controle, reprodutibilidade e portabilidade.

Você precisa criar um arquivo **Dockerfile** para definir as etapas de construção da imagem personalizada. O processo de construção é iniciado com o comando `docker build`, que lê o Dockerfile, executa as instruções nele contidas e gera uma nova imagem Docker personalizada. Depois, é possível executar um contêiner a partir dessa imagem usando o comando `docker run`.

Um **Dockerfile** é composto por uma série de instruções, como `FROM`, `RUN`, `COPY`, `CMD`, entre outras, que são executadas sequencialmente pelo mecanismo de construção do Docker. Na Listagem 4 é apresentado a sintaxe com as principais instruções de um Dockerfile. A ordem e o conteúdo dessas instruções impactam diretamente a eficiência do processo de construção e o tamanho final da imagem. Cada instrução representa uma etapa na construção da imagem e contribui para a criação de camadas (*layers*) que compõem a imagem final. Acesse https://docs.docker.com/develop/develop-images/dockerfile_best-practices/ para ver as melhores práticas para escrever Dockerfiles.

Listagem 4: Exemplo de um Dockerfile

```
# A instrução FROM especifica a imagem base a partir da qual a nova imagem será construída.  
FROM <imagem-base>  
# Define o diretório de trabalho para as instruções subsequentes, como RUN, CMD, COPY, etc.  
WORKDIR <diretorio-de-trabalho>  
# Executa comandos durante a construção da imagem. Ele é útil para instalar pacotes, configurar o ambiente,  
# etc. Cada comando RUN cria uma nova camada na imagem.  
RUN <comando-para-executar-durante-a-construcao>  
# Para copiar arquivos do host para o sistema de arquivos da imagem durante a construção.  
COPY <origem> <destino>  
# Define o comando padrão a ser executado quando um contêiner é iniciado a partir da imagem. Ele pode ser  
# sobrescrito no momento de execução do contêiner.  
CMD <comando-para-executar-quando-o-container-for-iniciado>
```

4.1 Criando uma imagem personalizada com um script de entrada

Nesse exemplo, iremos criar uma imagem personalizada a partir da imagem oficial do Ubuntu 24.04 LTS e instalaremos o aplicativo `figlet`, que é um programa que gera texto em arte ASCII. Depois, copiaremos um arquivo de texto com uma mensagem para dentro da imagem e criaremos um script de entrada (*entrypoint*) para exibir a mensagem usando o `figlet` quando o contêiner for executado.

```
mkdir lab && cd lab

echo "Oi Mundo" > mensagem.txt

echo "#!/bin/bash\ncat mensagem.txt | figlet" > entrypoint.sh

touch Dockerfile
```

Agora abra o diretório `lab` no Visual Studio Code (instale a extensão Container Tools) e edite o arquivo `Dockerfile` para que ele tenha o conteúdo conforme apresentado na listagem Listagem 6.

Listagem 6: Dockerfile com script de entrada

```
FROM ubuntu:24.04
WORKDIR /app

RUN apt-get update &&\
    DEBIAN_FRONTEND=noninteractive apt-get install -y figlet &&\
    rm -rf /var/lib/apt/lists/*

COPY mensagem.txt /app
COPY entrypoint.sh /app
RUN chmod +x entrypoint.sh

CMD ["/app/entrypoint.sh"]
```

O comando `RUN` é usado para atualizar a lista de pacotes disponíveis e instalar o aplicativo `figlet` dentro da imagem. O parâmetro `-y` é usado para confirmar automaticamente a instalação dos pacotes sem solicitar interação do usuário, o mesmo vale para a variável de ambiente `DEBIAN_FRONTEND=noninteractive`, que é usada para evitar que o processo de instalação solicite entradas interativas, como perguntas de configuração, durante a construção da imagem. Por fim, o comando `rm -rf /var/lib/apt/lists/*` é usado para limpar a cache de pacotes do `apt-get`, reduzindo o tamanho da imagem final.

Agora é necessário construir a imagem a partir do `Dockerfile` e depois executar um contêiner a partir dessa imagem para ver o resultado.

```
# Escolha o nome da imagem e informe o diretório onde está o Dockerfile (no caso, o diretório atual)
docker build -t minha-imagem .

# Executando um contêiner a partir da imagem criada
docker run --rm -it minha-imagem
```

5 Docker multistage

O Docker permite criar imagens de forma mais eficiente, utilizando o conceito de *multistage builds*. Isso significa que você pode dividir a construção da imagem em várias etapas, onde cada etapa pode usar uma imagem base diferente. Isso é útil para reduzir o tamanho final da imagem e melhorar a eficiência do processo de construção.

Para isso, você pode modificar o arquivo `Dockerfile` para incluir várias etapas de construção. Por exemplo, você pode usar uma imagem base leve para instalar as dependências e outra imagem base para executar a aplicação.

Crie um diretório chamado `exercicio-02` e dentro dele crie um projeto Java com o Gradle. Você pode usar o comando `gradle init` para criar um projeto básico. A estrutura do projeto deverá ser semelhante a apresentada na Listagem 8.

Listagem 8: Estrutura do projeto Java com Gradle

```
exercicio-02
|-- app
|   |-- build.gradle.kts
|   |-- src
|       |-- main
|           |-- java
|               |-- org
|                   |-- example
|                       |-- App.java
|               |-- resources
|       |-- test
|           |-- java
|               |-- org
|                   |-- example
|                       |-- AppTest.java
|               |-- resources
|-- Dockerfile
|-- gradle
|   |-- libs.versions.toml
|   |-- wrapper
|       |-- gradle-wrapper.jar
|       |-- gradle-wrapper.properties
|-- gradle.properties
|-- gradlew
|-- gradlew.bat
|-- settings.gradle.kts
```

Na raiz do projeto (diretório `exercicio-02`), crie um arquivo `Dockerfile` com o conteúdo conforme apresentado na listagem Listagem 9. Esse `Dockerfile` utiliza o conceito de *multistage builds* para compilar o projeto Java com Gradle e gerar uma imagem Docker enxuta, contendo apenas os arquivos necessários para executar a aplicação Java. Ou seja, a primeira etapa é responsável por compilar o projeto e gerar os arquivos de distribuição, enquanto a segunda etapa é responsável por criar a imagem final com apenas o JRE e os arquivos de distribuição.

Listagem 9: Dockerfile com multistage builds

```
# Usando uma imagem do Gradle 9.x com JDK 25 para compilar o projeto
FROM gradle:9.4.0-jdk25-alpine AS build
WORKDIR /codigo-java
# Copiando todo projeto gradle para dentro do contêiner
COPY . .
# Compilando o projeto e gerando os arquivos de distribuição (geralmente na pasta build/install/app)
RUN gradle installDist

#####
# Imagem final, contendo apenas o JRE e os arquivos de distribuição
#####
FROM eclipse-temurin:25-jre-alpine
WORKDIR /aplicacao
COPY --from=build /codigo-java/app/build/install/app /aplicacao
ENTRYPOINT ["./bin/app"]
```

Por fim, basta compilar a imagem a partir do Dockerfile e depois executar um contêiner a partir dessa imagem para ver o resultado.

```
docker build -t appjava .  
  
docker run --rm -it appjava
```

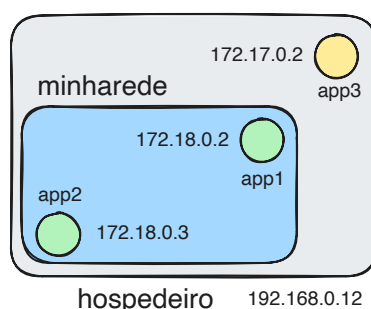
Nota

Use o comando `docker images` para ver o tamanho da imagem gerada. Você verá que a imagem final é muito menor do que a imagem da etapa de construção, pois ela contém apenas o JRE e os arquivos de distribuição, enquanto a etapa de construção contém o Gradle, o JDK e outros arquivos temporários usados durante a construção.

6 Contêineres e redes Docker

Ao executar aplicações em contêineres, é comum que elas precisem se comunicar entre si ou com o mundo exterior. Na arquitetura de microserviços, por exemplo, cada serviço é executado em um contêiner separado e eles precisam se comunicar para funcionar corretamente. O Docker oferece uma funcionalidade de redes que permite criar redes virtuais para conectar contêineres entre si e com o mundo exterior. Os contêineres conectados à mesma rede podem se comunicar usando seus nomes de host, o que facilita a configuração e a escalabilidade das aplicações.

Figura 3: Contêineres em redes Docker



Na Figura 3 é ilustrado um cenário com 3 contêineres, com os nomes `app1`, `app2` e `app3`. Os contêineres `app1` e `app2` estão conectados à mesma rede Docker, enquanto o contêiner `app3` está conectado a uma rede diferente. Isso significa que os contêineres `app1` e `app2` podem se comunicar entre si usando seus nomes de host, enquanto o contêiner `app3` não pode se comunicar diretamente com os outros dois contêineres, pois eles estão em redes diferentes.

Na Listagem 10 é apresentado um exemplo de como criar uma rede personalizada e conectar contêineres a essa rede.

Listagem 10: Criando uma rede personalizada e conectando contêineres

```
# Criando uma rede personalizada chamada minharede  
docker network create minharede  
  
# Executando dois contêineres e conectando-os à rede minharede  
docker run -it --name app1 --network minharede alpine ash  
docker run -it --name app2 --network minharede alpine ash  
  
# Executando um terceiro contêiner sem especificar a rede (ele ficará na rede padrão)  
docker run -it --name app3 alpine ash
```

Para testar a comunicação entre os contêineres, instale o pacote `iputils` em todos os contêineres para usar o comando `ping`. Será possível notar que os contêineres `app1` e `app2` conseguem se comunicar entre si usando seus nomes de host, enquanto o contêiner `app3` não consegue se comunicar com os outros dois contêineres, pois eles estão em redes diferentes.

```
# No terminal de cada contêiner, instale o pacote iputils para usar o comando ping
apk add --no-cache iputils

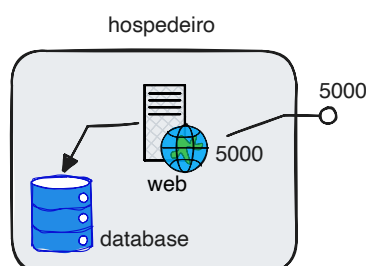
# No terminal do contêiner app1, execute o comando ping para o app2 usando o nome do host
ping -c 2 app2

# No terminal do contêiner app3, execute o comando ping para o app1 usando o nome do host
# Aqui não irá funcionar, pois o app3 está em uma rede diferente
ping -c 2 app1
```

7 Docker Compose

Quando temos um cenário com múltiplos contêineres, como uma aplicação web que depende de um banco de dados, por exemplo, pode ser trabalhoso gerenciar cada contêiner individualmente usando os comandos do Docker. O Docker Compose é uma ferramenta que facilita a definição e a execução de aplicações multi-contêineres. Ele permite que você defina os serviços, redes e volumes necessários para a aplicação em um arquivo YAML, chamado `compose.yaml`, e depois use um único comando para subir toda a aplicação.

Figura 4: Composição de contêineres com Docker Compose



Nessa seção iremos criar um cenário¹, ilustrado pela Figura 4, com uma aplicação web simples em Python que se comunica com um banco de dados Redis, ambos executando em contêineres separados, e usaremos o Docker Compose para facilitar a definição e a execução desses contêineres.

7.1 Criando a estrutura de diretórios e arquivos

Crie os diretórios e arquivos necessários para o exercício, conforme apresentado na listagem Listagem 12.

Listagem 12: Estrutura de diretórios e arquivos para o exercício

```
exercicio03
|-- app
|   |-- app.py
|-- compose.yaml
`-- Dockerfile
```

¹Adaptação da documentação oficial do Docker Compose, disponível em <https://docs.docker.com/compose/gettingstarted/>

Abra o diretório `exercicio03` no Visual Studio Code e edite os arquivos para que eles tenham o conteúdo conforme apresentado a seguir.

A aplicação web em Python é um servidor web simples usando o framework Flask, que se comunica com um banco de dados Redis para armazenar um contador. A cada vez que a página inicial é acessada, o contador é incrementado e o valor atualizado é exibido na página. Assim, precisamos de uma imagem com Python e que tenha instalado o flask e o cliente para Redis. Edite seu arquivo Dockerfile para que ele tenha o conteúdo conforme apresentado na listagem Listagem 13.

Listagem 13: Dockerfile para a aplicação python

```
FROM python:3.14-slim

WORKDIR /app

# Instalando as dependências no Python: flask e redis
RUN pip install --no-cache-dir flask redis

# Definindo variáveis de ambiente que serão usadas pela aplicação
ENV FLASK_APP=app.py
ENV FLASK_RUN_HOST=0.0.0.0

# Expondo a porta 5000 para acessar a aplicação Flask
EXPOSE 5000

# Comando para rodar a aplicação Flask
CMD ["flask", "run"]
```

A aplicação Python é composta por um único arquivo `app.py`, que contém o código para criar o servidor web usando Flask e se comunicar com o banco de dados Redis. Edite o arquivo `app/app.py` para que ele tenha o conteúdo conforme apresentado na listagem Listagem 14.

Listagem 14: Conteúdo do arquivo app.py

```
import redis
from flask import Flask

app = Flask(__name__)
# Conectando ao banco de dados Redis, usando o nome do serviço definido no compose.yaml
db = redis.Redis(host='database', port=6379)

def incrementa_contador():
    try:
        return db.incr('contador')
    except redis.exceptions.ConnectionError:
        return -1

@app.route('/')
def inicial():
    contador = incrementa_contador()
    return f'Valor do contador: {contador}.\n'
```

A composição é definida no arquivo `compose.yaml`, onde são descritos os serviços, redes e volumes necessários para a aplicação. Edite o arquivo `compose.yaml` para que ele tenha o conteúdo conforme apresentado na listagem Listagem 15.

No arquivo `compose.yaml` são definidos dois serviços: `web`, que contém a aplicação Python, e `database`, que é o banco de dados Redis. O serviço `web` é construído a partir do Dockerfile presente no diretório atual (indicado pela instrução `build: .`), enquanto o serviço `database` usa a imagem oficial do Redis disponível no Docker Hub.

O serviço `web` depende do serviço `database`, o que significa que o Docker Compose irá garantir que o serviço de banco de dados esteja em execução antes de iniciar a aplicação web. O volume mapeado para o serviço `web` permite que as alterações feitas no código fonte da aplicação Python

sejam refletidas imediatamente dentro do contêiner, facilitando o desenvolvimento e a depuração da aplicação. Além disso, a porta 5000 do contêiner da aplicação web é mapeada para a porta 5000 do hospedeiro, permitindo que você acesse a aplicação pela URL `http://localhost:5000`.

Listagem 15: Arquivo `compose.yaml`

```
services:
  web:
    build: .
    ports:
      - "5000:5000"
    volumes:
      - ./app:/app
    depends_on:
      - database
    environment:
      PYTHONUNBUFFERED: 1
      FLASK_ENV: development
  database:
    image: redis:alpine
```

Para executar a aplicação, é necessário estar no diretório onde está o arquivo `compose.yaml` e usar o comando `docker compose up`, que irá ler o arquivo de composição, construir as imagens necessárias (se ainda não estiverem construídas) e iniciar os contêineres de acordo com as definições do arquivo. O Docker Compose irá cuidar da criação da rede para os serviços se comunicarem entre si, do mapeamento de portas, do gerenciamento de volumes e da ordem de inicialização dos serviços com base nas dependências definidas.

7.2 Subindo várias instâncias de um mesmo contêiner

Imagine que você queira subir várias instâncias do serviço `web` para atender a um número maior de requisições. Para os usuários desses serviços deve-se deixar transparente o fato que existem múltiplas instâncias do serviço `web` rodando. Isso pode ser feito por meio de um proxy, como o NGINX, que seria mais um contêiner dentro da composição do Docker Compose.

Com o `docker compose` é simples subir múltiplas instâncias de um mesmo serviço. Por exemplo, se quiser subir 3 instâncias do serviço `web`, basta usar o comando `docker compose up --scale web=3`. O Docker Compose irá criar 3 contêineres separados para o serviço `web`, cada um com um nome único (como `web_1`, `web_2`, `web_3`) e irá gerenciar a comunicação entre eles e o serviço de banco de dados Redis.

Para esse cenário, iremos editar o arquivo `compose.yaml` para remover o mapeamento de portas do serviço `web`, pois o NGINX irá atuar como um proxy para encaminhar as requisições para as instâncias do serviço `web`. Assim, os usuários poderão acessar a aplicação pela porta do NGINX, sem se preocupar com as portas individuais de cada instância do serviço `web`. Edite seu arquivo `compose.yaml` para que ele tenha o conteúdo conforme apresentado na listagem Listagem 16.

Listagem 16: Arquivo `compose.yaml` com o Nginx

```
services:
  web:
    build: .
    volumes:
      - ./app:/app
    depends_on:
      - database
    environment:
      PYTHONUNBUFFERED: 1
      FLASK_ENV: development
  database:
    image: redis:alpine
```

```

nginx:
  image: nginx:alpine
  volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf
  depends_on:
    - web
  ports:
    - "4000:4000"

```

Na Listagem 17 é apresentado um exemplo de configuração do NGINX para atuar como proxy, onde ele escuta na porta 4000 e encaminha as requisições para o serviço web na porta 5000. O servidor de DNS interno do Docker se encarregará de fazer o balanceamento de carga entre as instâncias do serviço web, pois a resolução segue a estratégia de varredura cíclica (*round robin*).

Listagem 17: Configuração do NGINX para atuar como proxy

```

user  nginx;
events {
    worker_connections  500;
}
http {
    server {
        listen 4000;
        location / {
            proxy_pass http://web:5000;
        }
    }
}

```

Execute `docker compose up --scale web=3`, acesse a URL `http://localhost:4000`. Veja no console (onde executou o `docker compose`) qual instância de `web` atendeu o pedido. Pressione F5 no navegador e veja no console se outra instância atendeu o pedido.

Literatura recomendada

- Contêineres de software
 - www.redhat.com/pt-br/topics/containers/whats-a-linux-container
 - www.redhat.com/pt-br/topics/containers/what-is-docker
 - https://docs.docker.com/develop/develop-images/dockerfile_best-practices
- Microserviços
 - www.redhat.com/pt-br/topics/microservices/what-are-microservices
- *The Twelve-Factor App*
 - https://12factor.net/pt_br/